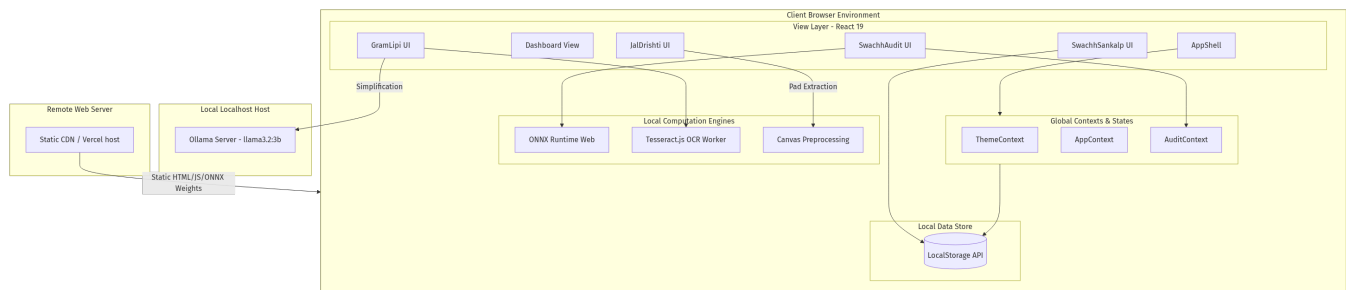


GramSetu AI - Software Architecture Specification

This document details the architectural layout, modules separation, global contexts, hooks pipelines, and data flow of GramSetu AI.

1. Overall System Architecture

GramSetu AI utilizes a local-first, zero-cloud architecture. It runs in-browser utilizing client hardware acceleration.



2. Module Sandboxing & Interactions

Each module inside the **app/** directory (such as **swachh-audit**, **jaldrishti**, **gramlipi**, and **swachhsankalp**) is structured as an isolated sandbox containing its own components, services, and hooks. Shared layouts and core utility functions reside at the root.

graph TD

```

subgraph Root Layout [Root Workspace]
  Components[Shared Components - components/ui, components/layout]
  Contexts[Shared Contexts - contexts/ThemeContext, contexts/AppContext]
  Utils[Shared Utilities - utils/cn, utils/generateId]
  Constants[Shared Constants - constants/routes]
end

subgraph Module Sandboxes [Module Boundaries]
  subgraph SwachhAudit [app/swachh-audit]
    AuditServices[services/detection]
    AuditHooks[hooks/useDetection]
    AuditComponents[components/DetectionPreview]
  end

  subgraph JalDrishti [app/jaldrishti]
    JalServices[services/imageProcessor]
    JalHooks[hooks/useStripAnalysis]
    JalComponents[components/WaterReportCard]
  end

  subgraph GramLipi [app/gramlipi]
    LipiServices[services/ocrService]
    LipiHooks[hooks/useDocumentOcr]
    LipiComponents[components/ocr-preview-card]
  end

  subgraph SwachhSankalp [app/swachhsankalp]
    SankalpServices[services/storage]
  end
end
```

```
        SankalpHooks[hooks/useSwachhData]
        SankalpComponents[components/WardLeaderboard]
    end
end

SwachhAudit --> Components
SwachhAudit --> Utils
SwachhAudit --> Contexts

JalDrishti --> Components
JalDrishti --> Utils

GramLipi --> Components
GramLipi --> Utils

SwachhSankalp --> Components
SwachhSankalp --> Utils
SwachhSankalp --> Constants
```

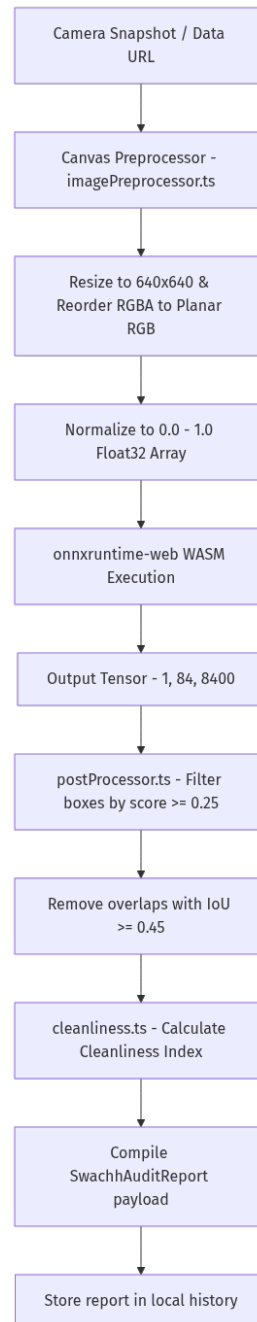
3. Data Flow & Local State Synchronization

Data persistence uses browser **localStorage**. Changes in UI components update the local React state, which is synchronously written to localStorage via hook wrappers.

```
graph TD
    UI[User Toggle Checklist / Join Campaign]
    --> StateUpdate[useSwachhData Hook State Updates]
    --> SaveStorage[storage.ts wrapper writes to localStorage]
    --> EventCheck[Trigger checkAchievements callback]
    --> Compare[Verify rules: tasks completed / campaigns joined]
    --> Unlock[Unlock Achievements Badge]
    --> SyncState[Set achievements state]
    --> Redraw[Update WardLeaderboard & score circle in UI]
```

4. AI Inference Execution Flow (SwachhAudit)

The pipeline for processing garbage audits starts at camera capture, flows through scale/normalization canvas layers, runs through on-device models, and filters outputs using Non-Maximum Suppression (NMS).



5. UI Component Hierarchy

The Root Layout renders the shell (**AppShell**) which houses navigation links, routing indicators, and the main page router.

graph TD

```

RootLayout[Root Layout - layout.tsx]
  --> AppShell[AppShell.tsx]

AppShell --> TopNavbar[TopNavbar.tsx]
AppShell --> Sidebar[Sidebar.tsx]
AppShell --> MobileNavigation[MobileNavigation.tsx]
AppShell --> MainContent[Main Content Router]
  
```

```
TopNavbar --> Breadcrumb[Breadcrumb.tsx]
TopNavbar --> ThemeToggle[ThemeToggle / Profile]

MainContent --> Dashboard[Dashboard - app/dashboard/page.tsx]
MainContent --> SwachhAuditPage[SwachhAudit - app/swachh-audit/page.tsx]
MainContent --> JalDrishtiPage[JalDrishti - app/jaldrishti/page.tsx]
MainContent --> GramLipiPage[GramLipi - app/gramlipi/page.tsx]
MainContent --> SwachhSankalpPage[SwachhSankalp - app/swachhsankalp/page.tsx]
```

6. Folder Dependency Flow

Dependencies flow from modular directories to root shared layers. Modules do not import from other modules.

